

We Trained a Domain Detector for Drones. One Class Collapsed to Zero.

Date: 2026-06-13

When you train a 9-class object detector on aerial imagery and get 89.4% mAP, the obvious interpretation is that the model works well. But numbers on a test set do not tell you what failed in the real world, and we discovered that one entire class had silently collapsed: the model never learned to detect landing pads.

Training a domain detector for drones means gathering aerial images, labeling objects that matter at altitude — other aircraft, people, landing zones, infrastructure — and fine-tuning a detection backbone. YOLOv8n seemed like the obvious choice, so we trained three iterations (v1, v2, v3) on progressively larger and more balanced datasets.

What the metrics said

V3 achieved 89.4% mAP on a held-out test split. The precision-recall curves looked healthy. Per-class performance appeared reasonable. All the usual metrics suggested a working detector.

What deployment revealed

The landing pad class (only 97 examples in the v3 training set) had effectively zero detection rate in practice. The model learned the visual features of other classes but never developed robust pad recognition. This was not a model capacity problem or an architecture problem — it was a data imbalance problem.

With only 97 pads against thousands of person and aircraft examples, the per-class loss was dominated by the larger classes. The optimizer pushed high-level accuracy by getting the common cases right and ignoring the tail class.

The fix

Class reweighting is the standard remedy: assign higher loss weight to the underrepresented class during training. For v3, we reweighted the landing pad class 5× and retrained. The updated model maintains 89.8% mAP overall while detecting landing pads at usable precision and recall.

The lesson: offline metrics on a balanced test set can hide catastrophic failure modes on the minority class in production. Always validate on the tail, and publish both aggregate metrics and per-class performance.